

TOOLS/SimplexNet 项目规划

面向强化学习的单纯形记忆网络工具包

Shuheng Zhang

2026 年 4 月 20 日

摘要

本文档描述 **TOOLS/SimplexNet** 项目的设计规划。该项目是一个面向强化学习的 Simplex Memory Network (SMN) 工具包, 基于 `exp0414_simplexNet` 的代码实现和 `writing/w0001-simplex-theory` 的理论基础。文稿分为三部分: 其一, 说明项目定位与目标用户; 其二, 描述两层类设计与两个子模块的组织架构; 其三, 给出实现优先级与 Phase 划分。本文档旨在在代码实现前完成设计对齐, 避免后续返工。

目录

1	项目定位	2
2	核心类设计	2
2.1	SMNmodule 类	2
2.2	SMN_RL 类	3
2.3	类继承关系	4
3	子模块设计	4
3.1	rl/ — 强化学习算法模块	4
3.2	tools/ — 工具模块	5
4	目录结构	6
5	实现优先级	7
5.1	Phase 1 (高优先级) — 最小可用版本	7
5.2	Phase 2 (中优先级) — 增强功能	7
5.3	Phase 3 (低优先级) — 完善生态	8
6	与 exp0414 的对比	8
7	下一步行动	8

1 项目定位

TOOLS/SimplexNet 的核心目标是提供一个**开箱即用**的 SMN 强化学习工具包，使研究者和开发者能够快速将 SMN 集成到 RL 系统中。与 exp0414_simplexNet 的”研究原型”定位不同，本项目的定位是**生产级工具包**。

目标用户包括：

- 想要快速使用 SMN 进行 RL 实验的研究者；
- 需要将 SMN 集成到现有强化学习系统中的开发者；
- 想要评估 SMN vs 传统 MLP 的算法工程师。

核心价值体现在三个方面：

1. **最小依赖**：仅需 PyTorch 和 Gymnasium，清晰接口；
2. **可恢复训练**：完整的 checkpoint 生命周期管理；
3. **向后兼容**：保持与 exp0414 的 API 互操作性。

2 核心类设计

本项目对外暴露**两个核心类**，分别对应两个独立的 Python 文件。

2.1 SMNmodule 类

SMNmodule 继承自 torch.nn.Module，纯网络前向传播模块。

```
class SMNmodule(nn.Module):
    """Simplex Memory Network as a PyTorch module.

    Args:
        n: Simplex dimension (n=2 -> triangle, n=3 -> tetrahedron)
        m: Resolution (lattice points per edge)
        n_in: Input dimension
        n_out: Output dimension
        activation: Hidden node activation
```

```

        x_bounds: Per-channel input normalization bounds

Example::
    # DQN Q-network (CartPole)
    q_net = SMNmodule(n=2, m=4, n_in=4, n_out=2)
    """

    def __init__(self, n=2, m=3, n_in=1, n_out=1,
                  activation='relu', x_bounds=None):
        super().__init__()
        # 初始化单纯形图、网络参数

    def forward(self, x: Tensor) -> Tensor:
        """Forward pass through the DAG."""

```

使用示例:

```

# DQN Q-network (CartPole)
q_net = SMNmodule(n=2, m=4, n_in=4, n_out=2)
output = q_net(state_tensor)

```

2.2 SMN_RL 类

SMN_RL 是 RL 功能封装类, 组合了网络、算法、持久化和 GUI。

```

class SMN_RL:
    """Reinforcement Learning with SMN.

    Attributes:
        module (SMNmodule): The underlying SMN network
        env (gym.Env): Gymnasium environment
        algorithm (str): 'DQN', 'PPO', 'REINFORCE'
        gamma (float): Discount factor
    """

```

```

Methods:
    train(episodes, render): Train the agent
    test(episodes): Evaluate the agent
    plot(save_path): Visualize training curves
    save_checkpoint(path): Save training state
    load_checkpoint(path): Resume from checkpoint
    launch_gui(): Open PySide interactive window
"""

```

使用示例:

```

# 快速使用
env = gym.make('CartPole-v1')
agent = SMN_RL.create_default(env, algorithm='DQN')
agent.train(episodes=1000)
agent.launch_gui() # 打开交互窗口

```

2.3 类继承关系

类	继承/组合关系
SMNmodule	继承 nn.Module
SMN_RL	组合 SMNmodule + gym.Env + 算法策略

3 子模块设计

项目包含两个子文件夹: rl/ 和 tools/。

3.1 rl/ — 强化学习算法模块

```

rl/
  __init__.py
  algorithms/

```

```
dqn.py      # DQN 算法
ppo.py      # PPO 算法 (Phase 2)
reinforce.py # REINFORCE 算法 (Phase 2)
mdp.py      # MDP 定义基类
env_wrapper.py # 环境封装
```

DQN 算法接口示例:

```
from rl.algorithms import DQN

dqn = DQN(
    q_network=smn_module,
    obs_dim=4, act_dim=2,
    gamma=0.99, lr=1e-3
)

dqn.select_action(state, epsilon=0.1)
dqn.store_transition(state, action, reward, next_state, done)
loss = dqn.train_step()
```

3.2 tools/ — 工具模块

```
tools/
__init__.py
checkpoint.py # CheckpointManager
logger.py    # TrainingLogger
plot.py      # 可视化函数
gui.py       # PySide 交互窗口
```

CheckpointManager 负责数据持久化:

```

from tools import CheckpointManager

ckpt_mgr = CheckpointManager('./checkpoints')
ckpt_mgr.save(module=smn, optimizer=opt, episode=100)
checkpoint = ckpt_mgr.load_latest()

```

TrainingLogger 负责追加式日志 (JSON Lines 格式):

```

from tools import TrainingLogger

logger = TrainingLogger('./logs')
logger.log('train_start', config={...})
logger.log('epoch', episode=1, reward=100, loss=0.5)

```

GUI 窗口提供 PySide 交互界面:

- 参数调节滑块 (n, m, lr, gamma, epsilon)
- 训练控制按钮 (Start, Pause, Resume, Stop)
- 实时曲线显示 (reward, loss, epsilon)
- Terminal log 面板
- Checkpoint 管理

4 目录结构

```

TOOLS/SimplexNet/
  SMNmodule.py          # 主脚本 1: SMNmodule 类
  SMN_RL.py             # 主脚本 2: SMN_RL 类
  rl/                   # 强化学习模块
    algorithms/
      mdp.py

```

```
env_wrapper.py
tools/                # 工具模块
    checkpoint.py
    logger.py
    plot.py
    gui.py
__init__.py           # 导出: SMNmodule, SMN_RL
README.md
examples/
    train_rl.py
    params.yaml
```

5 实现优先级

5.1 Phase 1 (高优先级) — 最小可用版本

- SMNmodule.py — 网络模型
- SMN_RL.py — RL 封装类
- rl/algorithms/dqn.py — DQN 算法
- tools/checkpoint.py — CheckpointManager
- tools/logger.py — TrainingLogger
- tools/plot.py — 训练曲线可视化
- __init__.py + README.md
- examples/train_rl.py — RL 训练示例

5.2 Phase 2 (中优先级) — 增强功能

- rl/algorithms/ppo.py — PPO 算法
- rl/algorithms/reinforce.py — REINFORCE 算法
- rl/mdp.py — MDP 定义基类
- tools/gui.py — PySide 交互窗口

5.3 Phase 3 (低优先级) — 完善生态

- tests/ — 单元测试
- examples/ — 更多示例
- 文档网站

6 与 exp0414 的对比

方面	exp0414 (现状)	SimplexNet (目标)
定位	研究原型 + 实验验证	生产级工具包
代码组织	扁平结构 (6 文件)	模块化分层
Checkpoint	仅内存 best_state	完整磁盘管理
日志	无	追加式训练日志
GUI	无	PySide 交互窗口

7 下一步行动

计划已就绪，等待用户指令开始 Phase 1 实现。实现顺序：

1. SMNmodule.py
2. rl/ 子文件夹
3. rl/algorithms/dqn.py
4. tools/ 子文件夹
5. tools/checkpoint.py
6. tools/logger.py
7. tools/plot.py
8. SMN_RL.py
9. __init__.py + README.md

8 参考文档

- exp0414 工程说明文稿：exp0414_simplexNet/report_tex/report.tex
- SMN 理论文档：writing/w0001-simplex-theory/report.tex

- exp0414 代码: `exp0414_simplexNet/src/`
- exp0420 RL 代码: `exp0420_RLforSMN/src/`